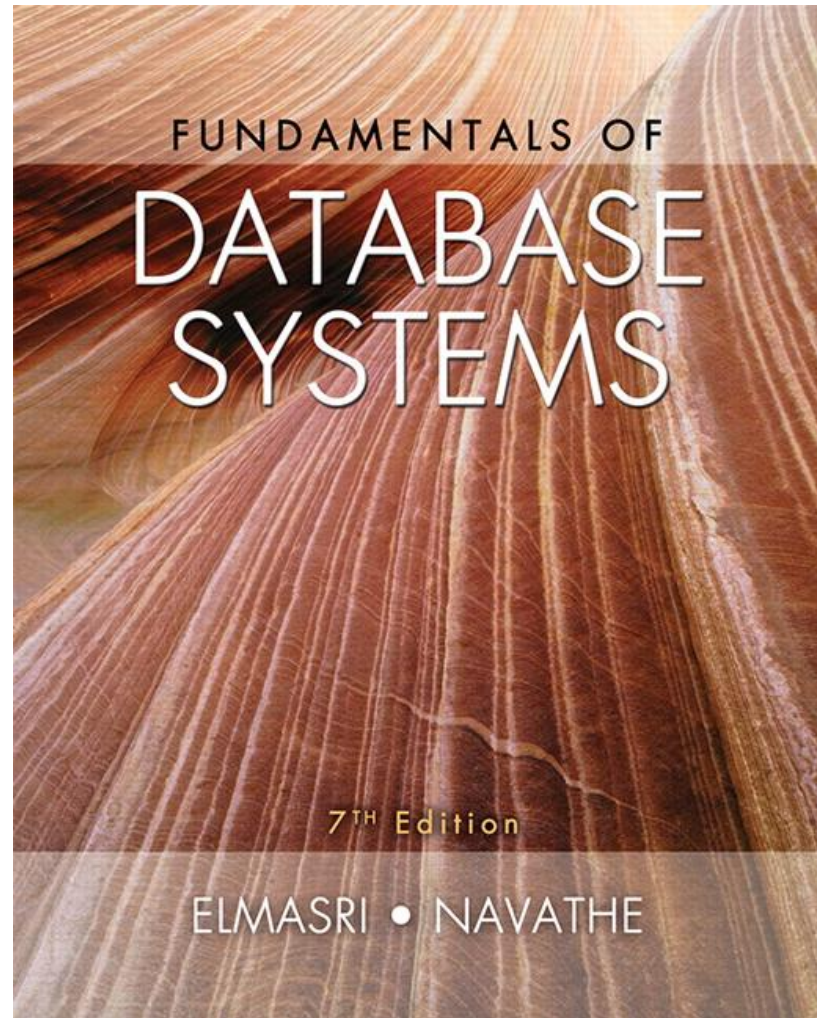




CHAPTER 18

Strategies for Query Processing

Agenda

- Session 1:
 - Give back exams.
 - Makeup.
 - Recording lectures.
 - Query Processing.
- Break
- Sessions 2:
 - Project progress presentations.

Introduction

- DBMS techniques to process a query
 - Scanner identifies query tokens
 - Parser checks the query syntax
 - Validation checks all attribute and relation names
 - Query tree (or query graph) created
 - Execution strategy or query plan devised
- Query optimization
 - Planning a good execution strategy

Query Processing

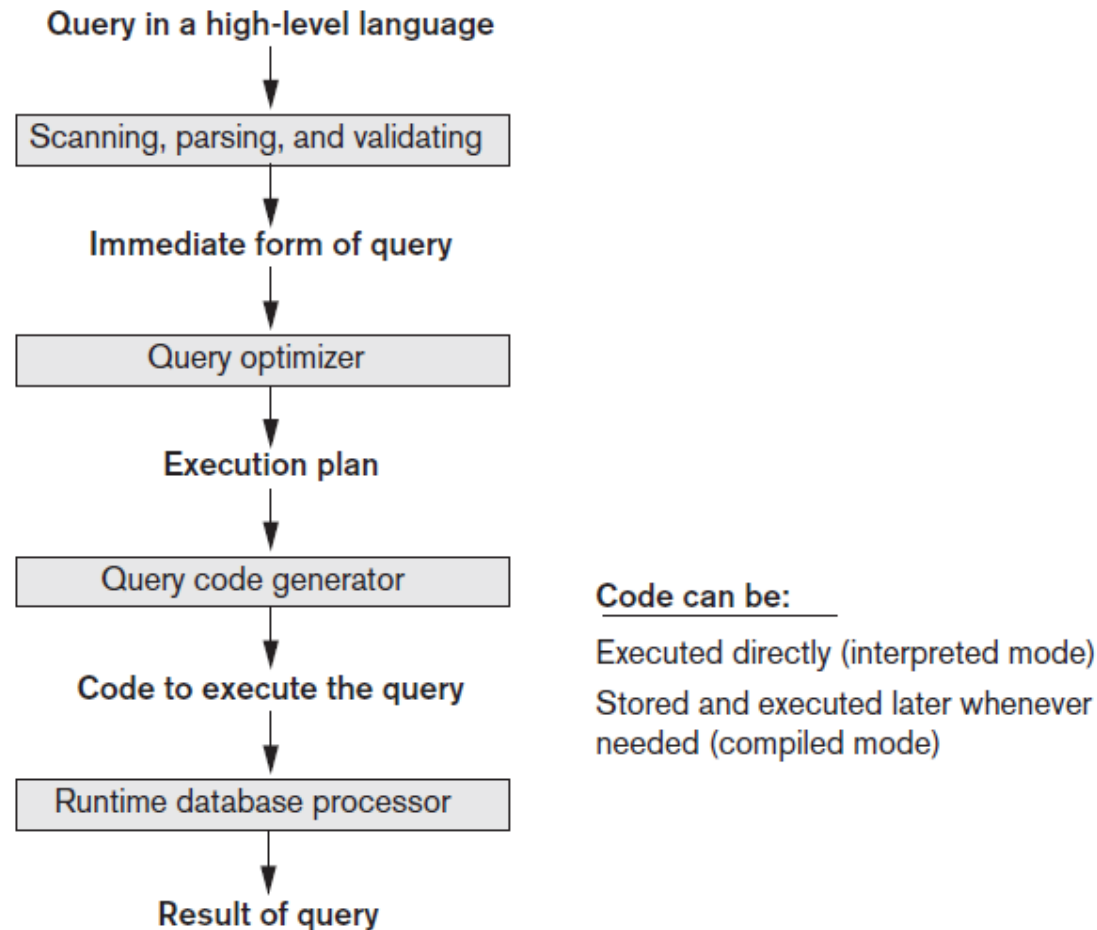


Figure 18.1 Typical steps when processing a high-level query

18.1 Translating SQL Queries into Relational Algebra and Other Operators

- SQL
 - Query language used in most RDBMSs
- Query decomposed into query blocks
 - Basic units that can be translated into the algebraic operators
 - Contains single SELECT-FROM-WHERE expression
 - May contain GROUP BY and HAVING clauses

Translating SQL Queries (cont'd.)

- Example:

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ( SELECT MAX (Salary)
                  FROM EMPLOYEE
                  WHERE Dno=5 );
```

- Inner block

```
( SELECT MAX (Salary)
  FROM EMPLOYEE
  WHERE Dno=5 )
```

- Outer block

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > c
```

Translating SQL Queries (cont'd.)

- Example (cont'd.)

- Inner block translated into:

$\mathcal{S}_{\text{MAX Salary}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$

- Outer block translated into:

$\pi_{\text{Lname, Fname}}(\sigma_{\text{Salary} > c}(\text{EMPLOYEE}))$

- Query optimizer chooses execution plan for each query block

Additional Operators

Semi-Join and Anti-Join

■ Semi-join

- Generally used for unnesting EXISTS, IN, and ANY subqueries
- Syntax: $T1.X \text{ } S = T2.Y$
 - T1 is the left table and T2 is the right table of the semi-join
- A row of T1 is returned as soon as T1.X finds a match with any value of T2.Y without searching for further matches

Unnesting Example

```
EMPLOYEE ( Ssn, Bdate, Address, Sex, Salary, Dno)  
DEPARTMENT ( Dnumber, Dname, Dmgrssn, Zipcode)
```

```
Q (SJ) : SELECT COUNT(*)  
FROM   DEPARTMENT D  
WHERE  D.Dnumber IN ( SELECT  E.Dno  
                      FROM    EMPLOYEE E  
                      WHERE   E.Salary > 200000)
```

```
SELECT COUNT(*)  
FROM    EMPLOYEE E, DEPARTMENT D  
WHERE   D.Dnumber S= E.Dno and E.Salary > 200000;
```

Semi-join



Additional Operators Semi-Join and Anti-Join (cont'd.)

■ Anti-join

- Used for unnesting NOT EXISTS, NOT IN, and ALL subqueries
- Syntax: $T1.x \text{ A } T2.y$
 - T1 is the left table and T2 is the right table of the anti-join
- A row of T1 is rejected as soon as T1.x finds a match with any value of T2.y
- A row of T1 is returned only if T1.x does not match with any value of T2.y

Unnesting Example

EMPLOYEE (Ssn, Bdate, Address, Sex, Salary, Dno)
DEPARTMENT (Dnumber, Dname, Dmgrssn, Zipcode)

```
Q (A) :  SELECT COUNT(*)  
FROM    EMPLOYEE  
WHERE   EMPLOYEE.Dno NOT IN (SELECT  DEPARTMENT.Dnumber  
                                FROM    DEPARTMENT  
                                WHERE   Zipcode =30332)
```

```
SELECT COUNT(*)  
FROM    EMPLOYEE, DEPARTMENT  
WHERE   EMPLOYEE.Dno A= DEPARTMENT AND Zipcode =30332
```

Anti-join



18.2 Algorithms for External Sorting

- Sorting is an often-used algorithm in query processing
 - Query with ORDER BY clause.
 - DISTINCT option in SELECT clause
 - Duplicate elimination in PROJECT operation
- External sorting
 - Algorithms suitable for large files that do not fit entirely in main memory
 - Sort-merge strategy based on sorting smaller subfiles (runs) and merging the sorted runs
 - Requires buffer space in main memory
 - DBMS cache

```

set    $i \leftarrow 1$ ;
       $j \leftarrow b$ ;           {size of the file in blocks}
       $k \leftarrow n_B$ ;         {size of buffer in blocks}
       $m \leftarrow \lceil (j/k) \rceil$ ; {number of subfiles- each fits in buffer}

```

{Sorting Phase}

```

while ( $i \leq m$ )
do {
    read next  $k$  blocks of the file into the buffer or if there are less than  $k$  blocks
    remaining, then read in the remaining blocks;
    sort the records in the buffer and write as a temporary subfile;
     $i \leftarrow i + 1$ ;
}

```

{Merging Phase: merge subfiles until only 1 remains}

```

set    $i \leftarrow 1$ ;
       $p \leftarrow \lceil \log_{k-1} m \rceil$  { $p$  is the number of passes for the merging phase}
       $j \leftarrow m$ ;
while ( $i \leq p$ )
do {
     $n \leftarrow 1$ ;
     $q \leftarrow \lceil j/(k-1) \rceil$ ; {number of subfiles to write in this pass}
    while ( $n \leq q$ )
    do {
        read next  $k-1$  subfiles or remaining subfiles (from previous pass)
        one block at a time;
        merge and write as new subfile one block at a time;
         $n \leftarrow n + 1$ ;
    }
     $j \leftarrow q$ ;
     $i \leftarrow i + 1$ ;
}

```

Figure 18.2

Outline of the sort-merge
algorithm for external sorting

Algorithms for External Sorting (cont'd.)

- Number of file blocks: b
 - number of disk blocks in the file
- Available buffer space: n_B ,
 - number of disk blocks that can fit in the buffer.
- Number of initial runs: n_R
- Degree of merging: d_M
 - Number of sorted subfiles that can be merged in each merge step
 - Smaller of $(n_B - 1)$ and n_R .
- Performance of the sort-merge algorithm
 - Number of disk block reads and writes before sorting is completed
 - $(2*b)$ sorting phase.
 - Once for reading and once for writing
 - Second term is for merging phase.

$$n_R = \lceil (b/n_B) \rceil$$

$$(2 * b) + (2 * b * (\log_{d_M} n_R))$$

18.3 Algorithms for SELECT Operation

- SELECT operation
 - Search operation to locate records in a disk file that satisfy a certain condition
 - File scan or index scan (if search involves an index)
- Search methods for simple selection
 - S1: Linear search (brute force algorithm)
 - Applies to any file
 - S2: Binary search
 - Requires file to be sorted on the search attribute.
 - S3a: Using a primary index
 - Requires the appropriate index to exist on the search attribute.
 - S3b: Using a hash key

Algorithms for SELECT Operation (cont'd.)

- Search methods for simple selection (cont'd.)
 - S4: Using a primary index to retrieve multiple records
 - Can retrieve records in a range.
 - S5: Using a clustering index to retrieve multiple records
 - S6: Using a secondary (B+ -tree) index on an equality comparison
 - Can retrieve records in a range.
 - S7a: Using a bitmap index
 - Suitable for retrievals where an attribute must match an enumerated set of values.
 - S7b: Using a functional index
 - Suitable when matching based on a function of one or more attributes on which a functional index exists.

Algorithms for SELECT Operation (cont'd.)

- Search methods for conjunctive (logical AND) selection
 - Using an individual index
 - Using a composite index
 - Intersection of record pointers
- Disjunctive (logical OR) selection
 - Harder to process and optimize

Algorithms for SELECT Operation (cont'd.)

Typical RDBMS catalog contents

For each relation (table) r with schema R containing r_R tuples:

- ❑ The number of rows/records or its cardinality: $|r(R)|$. We will refer to the number of rows simply as r_R .
- ❑ The “width” of the relation (i.e., the length of each tuple in the relation) this length of tuple is referred to as R .
- ❑ The number of blocks that relation occupies in storage: referred to as b_R .
- ❑ The blocking factor bfr , which is the number of tuples per block.

For each attribute A in relation R :

- ❑ The number of distinct values of A in R : $NDV(A, R)$.
- ❑ The max and min values of attribute A in R : $\max(A, R)$ and $\min(A, R)$.

Algorithms for SELECT Operation (cont'd.)

- Selectivity
 - Ratio of
 - the number of records (tuples) that satisfy the condition
 - to the total number of records (tuples) in the file
 - Number between
 - zero (no records satisfy condition) and
 - one (all records satisfy condition)
- Query optimizer receives input from system catalog to estimate approximate selectivity

18.4 Implementing the JOIN Operation

- JOIN operation

- One of the most time consuming in query processing
- EQUIJOIN (NATURAL JOIN)
- Two-way or multiway joins

$$R \bowtie_{A=B} S$$

OP6: EMPLOYEE $\bowtie_{Dno=Dnumber}$ DEPARTMENT
OP7: DEPARTMENT $\bowtie_{Mgr\ ssn=Ssn}$ EMPLOYEE

- Methods for implementing joins

- J1: Nested-loop join (nested-block join)
 - Brute force
 - For each record t in R (outer loop)
 - Retrieve every record s from S and (inner loop)
 - test whether the two records satisfy the join condition $t[A] == s[B]$
- J2: Index-based nested-loop join
 - Eliminate inner loop by using its index.
- J3: Sort-merge join
 - Sort files then scan both files concurrently in order of the join attribute.
- J4: Partition-hash join
 - Hash the smaller file into main memory
 - Use hashed version of every record in other file to look in the appropriate bucket in main memory.

Implementing the JOIN Operation (cont'd.)

```

(a) sort the tuples in  $R$  on attribute  $A$ ;           (*assume  $R$  has  $n$  tuples (records)*)
    sort the tuples in  $S$  on attribute  $B$ ;           (*assume  $S$  has  $m$  tuples (records)*)
    set  $i \leftarrow 1, j \leftarrow 1$ ;
    while  $(i \leq n)$  and  $(j \leq m)$ 
    do {
        if  $R(i)[A] > S(j)[B]$ 
            then set  $j \leftarrow j + 1$ 
        elseif  $R(i)[A] < S(j)[B]$ 
            then set  $i \leftarrow i + 1$ 
        else { (*  $R(i)[A] = S(j)[B]$ , so we output a matched tuple *)
            output the combined tuple  $\langle R(i), S(j) \rangle$  to  $T$ ;

            (* output other tuples that match  $R(i)$ , if any *)
            set  $l \leftarrow j + 1$ ;
            while  $(l \leq m)$  and  $(R(i)[A] = S(l)[B])$ 
            do { output the combined tuple  $\langle R(i), S(l) \rangle$  to  $T$ ;
                set  $l \leftarrow l + 1$ 
            }

            (* output other tuples that match  $S(j)$ , if any *)
            set  $k \leftarrow i + 1$ ;
            while  $(k \leq n)$  and  $(R(k)[A] = S(j)[B])$ 
            do { output the combined tuple  $\langle R(k), S(j) \rangle$  to  $T$ ;
                set  $k \leftarrow k + 1$ 
            }
            set  $i \leftarrow k, j \leftarrow l$ 
        }
    }
     $T \leftarrow R \bowtie_{A=B} S.$ 

```

Figure 18.3 Implementing JOIN, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE by using sort-merge, where R has n tuples and S has m tuples. (a) Implementing the JOIN operation

Implementing the PROJECT Operation (cont'd.)

$$T \leftarrow \pi_{\langle \text{attribute list} \rangle}(R).$$

```
(b) create a tuple  $t[\langle \text{attribute list} \rangle]$  in  $T'$  for each tuple  $t$  in  $R$ ;  
    (*  $T'$  contains the projection results before duplicate elimination *)  
    if  $\langle \text{attribute list} \rangle$  includes a key of  $R$   
    then  $T \leftarrow T'$   
    else { sort the tuples in  $T'$ ;  
        set  $i \leftarrow 1, j \leftarrow 2$ ;  
        while  $i \leq n$   
        do { output the tuple  $T'[i]$  to  $T$ ;  
            while  $T'[i] = T'[j]$  and  $j \leq n$  do  $j \leftarrow j + 1$ ;          (* eliminate duplicates *)  
             $i \leftarrow j; j \leftarrow i + 1$   
        }  
    }  
    (*  $T$  contains the projection result after duplicate elimination*)
```

Figure 18.3 (cont'd.) Implementing JOIN, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE by using sort-merge, where R has n tuples and S has m tuples.
(b) Implementing the PROJECT operation

Implementing the JOIN Operation (cont'd.)

$$T \leftarrow R \cup S.$$

```
(c) sort the tuples in  $R$  and  $S$  using the same unique sort attributes;  
    set  $i \leftarrow 1, j \leftarrow 1$ ;  
    while  $(i \leq n)$  and  $(j \leq m)$   
    do { if  $R(i) > S(j)$   
        then { output  $S(j)$  to  $T$ ;  
              set  $j \leftarrow j + 1$   
            }  
        elseif  $R(i) < S(j)$   
        then { output  $R(i)$  to  $T$ ;  
              set  $i \leftarrow i + 1$   
            }  
        else set  $j \leftarrow j + 1$  (*  $R(i)=S(j)$ , so we skip one of the duplicate tuples *)  
    }  
    if  $(i \leq n)$  then add tuples  $R(i)$  to  $R(n)$  to  $T$ ;  
    if  $(j \leq m)$  then add tuples  $S(j)$  to  $S(m)$  to  $T$ ;
```

Figure 18.3 (cont'd.) Implementing JOIN, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE by using sort-merge, where R has n tuples and S has m tuples.
(c) Implementing the UNION operation

Implementing the JOIN Operation (cont'd.)

$$T \leftarrow R \cap S.$$

```
(d) sort the tuples in  $R$  and  $S$  using the same unique sort attributes;  
    set  $i \leftarrow 1, j \leftarrow 1$ ;  
    while ( $i \leq n$ ) and ( $j \leq m$ )  
    do { if  $R(i) > S(j)$   
        then set  $j \leftarrow j + 1$   
        elseif  $R(i) < S(j)$   
        then set  $i \leftarrow i + 1$   
        else { output  $R(j)$  to  $T$ ;           (*  $R(i) = S(j)$ , so we output the tuple *)  
              set  $i \leftarrow i + 1, j \leftarrow j + 1$   
        }  
    }
```

Figure 18.3 (cont'd.) Implementing JOIN, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE by using sort-merge, where R has n tuples and S has m tuples.
(d) Implementing the INTERSECTION operation

Implementing the JOIN Operation (cont'd.)

$$T \leftarrow R - S.$$

```
(e) sort the tuples in  $R$  and  $S$  using the same unique sort attributes;  
    set  $i \leftarrow 1, j \leftarrow 1$ ;  
    while  $(i \leq n)$  and  $(j \leq m)$   
    do { if  $R(i) > S(j)$   
        then set  $j \leftarrow j + 1$   
        elseif  $R(i) < S(j)$   
        then { output  $R(i)$  to  $T$ ;          (*  $R(i)$  has no matching  $S(j)$ , so output  $R(i)$  *)  
            set  $i \leftarrow i + 1$   
        }  
        else set  $i \leftarrow i + 1, j \leftarrow j + 1$   
    }  
    if  $(i \leq n)$  then add tuples  $R(i)$  to  $R(n)$  to  $T$ ;
```

Figure 18.3 (cont'd.) Implementing JOIN, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE by using sort-merge, where R has n tuples and S has m tuples.
(e) Implementing the DIFFERENCE operation

Implementing the JOIN Operation (cont'd.)

- Available buffer space has important effect on some JOIN algorithms
- Nested-loop approach
 - Read as many blocks as possible at a time into memory from the file whose records are used for the outer loop
 - Advantageous to use the file with fewer blocks as the outer-loop file

Implementing the JOIN Operation (cont'd.)

■ Join selection factor

- Fraction of records in one file that will be joined with records in another file
- Depends on the particular equijoin condition with another file
- Affects join performance
- e.g. department and employees tables
 - Reading records at index of each employee SSN to find managers of a Department
 - vs. reading records of each department to find employee who has that SSN in its index.

■ Partition-hash join

- Each file is partitioned into M partitions using the same partitioning hash function on the join attributes
- Each pair of corresponding partitions is joined

Implementing the JOIN Operation (cont'd.)

- Hybrid hash-join
 - Variation of partition hash-join
 - Joining phase for one of the partitions is included in the partition
 - Goal: join as many records as possible during the partitioning phase to save cost of storing records on disk and then rereading during the joining phase

18.5 Algorithms for PROJECT and Set Operations

- PROJECT operation
 - After projecting R on only the columns in the list of attributes, any duplicates are removed by treating the result strictly as a set of tuples
- Default for SQL queries
 - No elimination of duplicates from the query result
 - Duplicates eliminated only if the keyword DISTINCT is included

Algorithms for PROJECT and Set Operations (cont'd.)

- Set operations
 - UNION
 - INTERSECTION
 - SET DIFFERENCE
 - CARTESIAN PRODUCT
- Set operations sometimes expensive to implement
 - Sort-merge technique
 - Hashing

Algorithms for PROJECT and Set Operations (cont'd.)

- Use of anti-join for SET DIFFERENCE
 - UNION, INTERSECTION and EXCEPT
 - traditional sets: no duplicate records in result.
 - EXCEPT or MINUS in SQL
 - Example: Find which departments have no employees

Select Dnumber from DEPARTMENT MINUS Select Dno from EMPLOYEE;

becomes

```
SELECT DISTINCT DEPARTMENT.Dnumber
FROM   DEPARTMENT, EMPLOYEE
WHERE  DEPARTMENT.Dnumber < EMPLOYEE.Dno
```


Algorithms for PROJECT and Set Operations (cont'd.)

- Use of select-join for ALL
 - UNION ALL, INTERSECT ALL and EXCEPT ALL
 - multisets
 - INTERSECT ALL in SQL
 - Example: Find all departments which have employees working on a project controlled by the department

```
SELECT Dno from EMPLOYEE  
INTERSECT ALL  
SELECT Dum from PROJECT
```

- becomes

```
SELECT DISTINCT EMPLOYEE.Dno  
FROM DEPARTMENT, EMPLOYEE  
WHERE EMPLOYEE.Dno S = DEPARTMENT.Dnumber
```

18.6 Implementing Aggregate Operations and Different Types of JOINS

- Aggregate operators

- MIN, MAX, COUNT, AVERAGE, SUM
- Can be computed by a table scan or using an appropriate index

- Example:

```
SELECT  MAX(Salary)
FROM    EMPLOYEE;
```

- If an (ascending) B+ -tree index on Salary exists:
 - Optimizer can use the Salary index to search for the largest Salary value
 - Follow the rightmost pointer in each index node from the root to the rightmost leaf

Implementing Aggregate Operations and Different Types of JOINS (cont'd.)

■ AVERAGE or SUM

- Index can be used if it is a dense index
- Computation applied to the values in the index
- Nondense index can be used if actual number of records associated with each index value is stored in each index entry

■ COUNT

- Number of values can be computed from the index

Implementing Aggregate Operations and Different Types of JOINS (cont'd.)

- Standard JOIN (called INNER JOIN in SQL)
- Variations of joins

- Outer join

- Left, right, and full
- Example:

```
SELECT E.Lname, E.Fname, D.Dname  
FROM (EMPLOYEE E LEFT OUTER JOIN DEPARTMENT D ON E.Dno = D.Dnumber);
```

- Semi-Join
- Anti-Join
- Non-Equi-Join:
 - Join when entries are NOT equal

18.7 Combining Operations Using Pipelining

- SQL query translated into relational algebra expression
 - Sequence of relational operations
- Materialized evaluation
 - Creating, storing, and passing temporary results
- General query goal: minimize the number of temporary files
- Pipelining or stream-based processing
 - Combines several operations into one
 - Avoids writing temporary files

Combining Operations Using Pipelining (cont'd.)

- Pipelined evaluation benefits
 - Avoiding cost and time delay associated with writing intermediate results to disk
 - Being able to start generating results as quickly as possible

■ Example: $\pi_{name}(\sigma_{dno=5}(EMPLOYEE))$

Without Pipelining

1. Compute:

$\sigma(dno=5)$

→ store intermediate table

2. Apply:

$\pi(name)$

→ read from stored result

With Pipelining

- Read each tuple from EMPLOYEE
- Immediately:
 - Check `dno = 5`
 - If true → output `name`

- ✓ No intermediate storage
- ✓ Done in one pass

Combining Operations Using Pipelining (cont'd.)

- Pipelined evaluation benefits
 - Avoiding cost and time delay associated with writing intermediate results to disk
 - Being able to start generating results as quickly as possible
- Example:

```
SELECT e.name  
FROM employee e  
JOIN department d  
ON e.dno = d.dnumber  
WHERE d.dname = 'Research';
```

With Pipelining:

- Read employee row
 - Join with matching department
 - Apply WHERE condition
 - Output name immediately
- ✓ No temporary tables needed

When is Pipelining Possible

✓ Works well for:

- Selection (σ)
- Projection (π)
- Simple joins

✗ Harder for:

- Sorting
- Aggregation (GROUP BY)
- Duplicate elimination

Combining Operations Using Pipelining (cont'd.)

- Iterator
 - Operation implemented in such a way that it outputs one tuple at a time
 - Many iterators may be active at one time
- Iterator interface methods
 - Open()
 - Get_Next()
 - Close()
- Some physical operators may not lend themselves to the iterator interface concept
 - Pipelining not supported
- Iterator concept can also be applied to access methods

18.8 Parallel Algorithms for Query Processing

- Parallel database architecture approaches
 - Shared-memory architecture
 - Multiple processors can access common main memory region
 - Shared-disk architecture
 - Every processor has its own memory
 - Machines have access to all disks
 - Shared-nothing architecture
 - Each processor has own memory and disk storage
 - Most used in parallel database systems

Parallel Algorithms for Query Processing (cont'd.)

- Linear speed-up
 - Linear reduction in time taken for operations
- Linear scale-up
 - Constant sustained performance by increasing the number of processors and disks

Parallel Algorithms for Query Processing (cont'd.)

- Operator-level parallelism
 - Horizontal partitioning: n processors
 - Round-robin partitioning:
 - assign i^{th} Tuple to processor $i \% n$
 - Range partitioning:
 - Divide the data into n ranges of almost equal size
 - Assign each range to a processor
 - Hash partitioning
 - Hash data into n buckets.
 - Assign each bucket to a processor.
 - Sorting
 - If data has been range-partitioned on an attribute:
 - Each partition can be sorted separately in parallel
 - Results concatenated
 - Reduces sorting time

Parallel Algorithms for Query Processing (cont'd.)

- Selection
 - If condition is an equality condition on an attribute used for range partitioning:
 - Perform selection only on partition to which the value belongs
- Projection without duplicate elimination
 - Perform operation in parallel as data is read
- Duplicate elimination
 - Sort tuples and discard duplicates

Parallel Algorithms for Query Processing (cont'd.)

- Parallel joins divide the join into n smaller joins
 - Perform smaller joins in parallel on n processors
 - Take a union of the result
- Parallel join techniques
 - Equality-based partitioned join
 - Inequality join with partitioning and replication
 - Parallel partitioned hash join

Parallel Algorithms for Query Processing (cont'd.)

- Aggregation
 - Achieved by partitioning on the grouping attribute and then computing the aggregate function locally at each processor
- Set operations
 - If argument relations are partitioned using the same hash function, they can be done in parallel on each processor

Parallel Algorithms for Query Processing (cont'd.)

- Intraquery parallelism
 - Approaches
 - Use parallel algorithm for each operation, with appropriate partitioning of the data input to that operation
 - Execute independent operations in parallel
- Interquery parallelism
 - Execution of multiple queries in parallel
 - Goal: scale up
 - Difficult to achieve on shared-disk or shared-nothing architectures

18.9 Summary

- SQL queries translated into relational algebra
- External sorting
- Selection algorithms
- Join operations
- Combining operations to create pipelined execution
- Parallel database system architectures